

Dark Matter Physics Simulation & Analysis for the Dark Photon Milestone 4

By: Nikhil Chaba, Samuel Rock, and Jacob Woods

Weizsacker Williams approximation codebase

- Each class definition takes "here's a beam particle and a target," and produces everything downstream code needs, the target's mass and charge shape, and the collision re-described from the target's own resting point of view, so the rest of the notebook never has to think about labs, beams, or reference frames again.

Experiment class definition

```
class Experiment():  
    def __init__(self, name, li, E_li, Z, A, E_nuc = None, coherent = True):
```

- This class definition is the data container/kinematics calculator for “a beam of leptons hitting the nucleus”

Four Experiment Class definitions declared

- Four of these class definitions are declared
- EIC, muon-beam-dump MuBeD, EIC_far/back

```
#Let's add the EIC in Gold ( $Z = 79$ ,  $A = 197$ ) mode and a lead ( $Z = 82$ ,  $A = 208$ ) Muon Beam Dump (MuBeD).
```

```
EIC = Experiment('EIC', 'e', 18, 79, 197, E_nuc = 110 * 197, coherent = True)
MuBeD = Experiment('MuBeD', 'mu', 1000, 82, 208, coherent = False) # Don't care about coherent production for beam-dump scenario
EIC_far = Experiment('EIC_new', 'e', 18, 79, 197, E_nuc = 110 * 197, coherent = True)
EIC_back = Experiment('EIC_back', 'e', 18, 79, 197, E_nuc = 110 * 197, coherent = True)
```

Cross Section Calculation

- Wrapper function
- Is used throughout the rest of the notebook: given an Experiment, a mass m , an optional integration kernel, and a coupling g , it returns the total kernel-weighted production cross-section in the wanted/requested units
- Performs nested 2D trapezoidal integration and integrated the resulting 1D array over the angle of the beam axis of the produced dark boson getting to down to a singular cross-section value

```
def cross_section(experiment, m, kernel = None, n_pts_E = 100, n_pts_th = 200, units = 'pb', g = 1):
    ...
    assert units in ['GeV', 'pb', 'fb']

    # Ensure the cross-section is cached for units = 'GeV', so computing with different units leads to no
    # extra computational time.
    if units == 'fb':
        | return hc2_fbGeV2*cross_section(experiment, m, kernel, n_pts_E, n_pts_th, units = 'GeV', g = g)

    if units == 'pb':
        | return cross_section(experiment, m, kernel, n_pts_E, n_pts_th, units = 'fb', g = g)/1000

    # Ensure the cross-section is cached for g = 1, so computing with another g leads to no
    # extra computational time.
    if g != 1:
        | return g**2 * cross_section(experiment, m, kernel, n_pts_E, n_pts_th, units)

    Ek, th_k, dcx = vector_production_differential_cross_section(experiment, m, n_pts_E, n_pts_th)

    if not kernel:
        | kernel = lambda x, y, z: 1

    return np.trapezoid(np.trapezoid(kernel(m, Ek.reshape(-1, 1), th_k.reshape(1, -1)) * dcx, x = Ek, axis = 0), x = th_k, axis = 0)
```

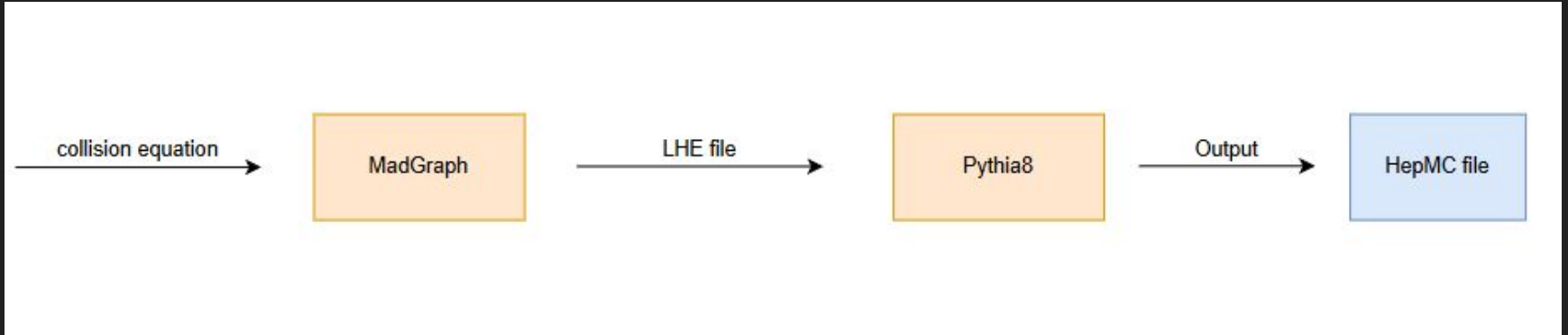
Exploring a Gold collision

- Originally our pipeline software could not run a gold -e collision simulation completely through and return a hepmc file.
- We need to run a heavy ion and electron collision.
- Troubleshooting the heavy ion and electron collision involved exploring the roadblock, Pythia8.



Exploring Pythia8

- Pythia8 is built upon many different scripts.
- Our objective concerned two main scripts.



Main121.cc

- Creates the Pythia event generator
- Sets the frame type to 4, only takes input from LHEF
- Reads a hardcoded LHEF
- Creates a histogram, as it reads events, its looking for particles that cannot decay any further and carry a charge
- Outputs the final histogram

```
25 bool useGzip = false;
26 #endif
27 cout << " useGzip = " << useGzip << endl; // Can also read a gzipped LHE file (LHEF)
28
29 // Generator, We here stick with default values, but changes
30 // could be inserted with readString or readFile.
31 Pythia pythia; // Creates Pythia Generator, events will come from LHEF
32
33 // Initialize Les Houches Event File run. List initialization information.
34 pythia.readString("Beams:frameType = 4");
35 // 5 Different frameTypes, this one means that Pythia reads incoming events and beam definitions directly from LHEF
36 if (useGzip) pythia.readString("Beams:LHEF = ttbar.lhe.gz"); // Hardcodes the file, can just change it
37 else pythia.readString("Beams:LHEF = ttbar.lhe");
38 // This initializes Pythia, If Pythia fails to initialize, the program exits with error.
39 if (!pythia.init()) return 1;
40
41 // Book histogram, 100 bins covering 0-399 charged particles
42 Hist nCharged("charged particle multiplicity",100,-0.5,399.5);
43
44 // Allow for possibility of a few faulty events.
45 int nAbort = 10; // Maximum number of allowed consecutive event failures before program termination
46 int iAbort = 0; // Local counter variables to keep track of how many consecutive failures have occurred
47
48 // Begin event loop; generate until none left in input file.
49 while (iAbort < nAbort) {
50
51     // Generate events and process to the next one, and check whether generation failed.
52     if (!pythia.next()) {
53
54         // If failure because reached end of file then exit event loop.
55         if (pythia.info.atEndOfFile()) break;
56         ++iAbort;
57         continue;
58     }
59
60     // Sum up final charged multiplicity and fill in histogram.
61     int nChg = 0;
62     for (int i = 0; i < pythia.event.size(); ++i)
63         if (pythia.event[i].isFinal() && pythia.event[i].isCharged()) // isFinal returns boolean, means that it does not decay further
64             ++nChg; // isCharged returns boolean, means that it carries a charge
65     nCharged.fill(nChg); // Number of charged final particles
66
67 // End of event loop.
68 }
69
70 // Give statistics. Print histogram.
71 pythia.stat(); // Standard function called at the end of a sim run to print statistics summary to console/cml
72 cout << nCharged; // Prints histogram, nCharged in Hist class
73
74 // Done.
75 return 0;
76 }
```

Main131.cc

- Beams:eCM = 8000.: 8 TeV collision energy
- HardQCD:all = on : hard QCD scattering
- PhaseSpace:pTHatMin = 20.: 20 GeV minimum scattering scale
- 100 attempts
- Skip if event fails

```
Pythia pythia;
pythia.readString("Beams:eCM = 8000.");
pythia.readString("HardQCD:all = on");
pythia.readString("PhaseSpace:pTHatMin = 20.");

// If Pythia fails to initialize, exit with error.
if (!pythia.init()) return 1;

Hist mult("charged multiplicity", 100, -0.5, 799.5);

// Begin event loop. Generate event. Skip if error.
for (int iEvent = 0; iEvent < 100; ++iEvent) {
    if (!pythia.next()) continue;
```


- Hard quark/gluon collision
- Parton shower and hadronization
- Particle decays
- Final event record
- Count qualifying particles per event
- Add each count to a histogram
- Show the distribution across events

```
• // Find number of all final charged particles and fill histogram.  
• int nCharged = 0;  
• for (int i = 0; i < pythia.event.size(); ++i)  
• | if (pythia.event[i].isFinal() && pythia.event[i].isCharged())  
• | | ++nCharged;  
• mult.fill(nCharged);
```

Converting hepmc version 3 to version 2

- `python3 -m pip install pyhepmc`

```
• // Interface for conversion from Pythia8::Event to HepMC event.  
• // Specify file where HepMC events will be stored.  
• Pythia8ToHepMC toHepMC("main131.hepmc");
```

```
• // Interface for conversion from Pythia8::Event to HepMC event.  
• // Specify file where HepMC events will be stored.  
• Pythia8ToHepMC toHepMC("main131.hepmc", Pythia8ToHepMC::ascii2);
```

Converting an lhe file into a hepmc file

- Getting Main131.cc to take in an lhe file only took a few lines to be amended
- The problem we ran into was getting Pythia8 to take in gold ion as a collision parameter
- We needed a workaround.

```
PYTHIA Error in ResonanceWidths::init: resonance mass too small for id = 1023
PYTHIA Error in BeamSetup::checkBeams: cannot handle this beam combination
PYTHIA Abort from BeamSetup::initBeams: checkBeams initialization failed
```

Pythia8 CheckBeam Solution

- Changing the gold ion PDG id into a photon PDG id
- It is nearly the same physics

```
<init>
```

```
1000791970 11 2.167000e+04 1.800000e+01 0 0 0 0 -4 1  
3.934400e+04 9.701687e+01 3.934400e+04 1
```

```
<init>
```

```
22 11 2.167000e+04 1.800000e+01 0 0 0 0 -4 1  
3.934400e+04 9.701687e+01 3.934400e+04 1
```

Milestone 4 Task Matrix

Task	Completion:	Nikhil	Sam	Jacob	To do:
Understand the Weizsacker-Williams approximation code base	95%	-	1/2	1/2	We still need to confirm this is the correct and wanted approach from our client/advisor.
Test the Weizsacker-Williams approximation code base	0%	1/3	1/3	1/3	We will still need to confirm this is the correct approach before implementing this ourselves.
Explore the process of a gold -e collision	40%	1/3	1/3	1/3	We have the initial simulation running of a gold collision, but we want to be able to run our scripts on the data output to format the data in a readable way.
Write a user manual	10%	1/3	1/3	1/3	We have to write the processes and steps required for later parts of our updated pipeline

Milestone 5

- Task 1: Confirm that the Weizsacker algorithm is the correct path for us to follow.
- Task 2: Testing the Weizsacker-Williams approximation codebase (contingent on the confirmation this is the correct path)
- Task 3: Finish a gold e^- collision analysis in madgraph and compare it to $p e^- > p e^- \chi p$ collision
- Task 4: Continue writing the user manual, focusing on the later parts of our updated pipeline

Milestone 5 Task Matrix

Task	Nikhil	Sam	Jacob
Confirm that the Weizsacker algorithm is the correct path for us to follow.	-	1/2	1/2
Testing the Weizsacker-Williams approximation codebase (contingent on the confirmation that this is the correct path)	1/3	1/3	1/3
Analyze a gold e- collision at the reconstruction/root level and compare it to $p e^- \rightarrow p e^- \chi p$ collision	1/3	1/3	1/3
Write user manual	1/3	1/3	1/3

Thank You